MEDICARD / ENGINEERING REFERENCE

Admin Dashboard Documentation

Architecture, API integration, authentication, features, and development workflows

Compiled from 11 repository Markdown files

14 October 2026

NAVIGATION

Contents

Select a chapter to jump to it in the PDF. Documentation and route links stay inside this PDF; source paths are plain text.

01 Project overview →

02 Architecture and project structure →

03 Development setup and deployment →

04 Routing and layouts →

05 Authentication and permissions →

06 API integration and request inventory →

07 Forms and UI states →

08 Providers and map →

09 Clients and operations →

10 Staff and notifications →

11 Catalog and content →

CHAPTER 01

MediCard Admin Dashboard

A React and Vite administration portal for providers, clients, invoices, tickets, card operations, staff, notifications, reference data, and site content. The interface contains English and Arabic fields; individual Arabic content is displayed with right-to-left direction where needed.

Start locally

Use Node.js 22.x and npm.

```bash
npm ci
npm run dev
```

The API URL is currently defined in `src/shared/api/axiosInstance.js`. No project-specific Vite environment variable is referenced in the source. See development setup before connecting to an environment.

Scripts

Command	Purpose
`npm run dev`	Start the Vite development server
`npm run build`	Build the static app into `dist/`
`npm run preview`	Preview a production build locally
`npm run lint`	Run ESLint

The repository has no test script.

Documentation

- Architecture and project structure
- Development setup and deployment
- Routes and layouts
- Authentication and permissions
- API integration and endpoint inventory
- Forms and UI states

- Providers and map
- Clients and operations
- Staff and notifications
- Catalog and content

These guides describe the checked-in front-end implementation. An endpoint or response shape shown in client code does not, by itself, verify the live backend contract.

CHAPTER 02

Architecture and project structure

README · Routing · API integration

Runtime composition

`src/main.jsx` mounts `App` in React Strict Mode and a React Hot Toast `Toaster`. `src/app/App.jsx` nests `QueryClientProvider`, `AuthProvider`, `Suspense`, and `RouterProvider`. The Query client [src/app/queryClient.js] uses TanStack Query defaults; no global retry or stale-time policy is configured there.

MERMAID DIAGRAM SOURCE

```
flowchart TD
  Main["main.jsx: React root and Toaster"] --> App["App.jsx"]
  App --> Query["QueryClientProvider"]
  Query --> Auth["AuthProvider"]
  Auth --> Router["RouterProvider"]
  Router --> Guard["ProtectedRoute"]
  Guard --> Layout["DashboardLayout"]
  Layout --> Route["PermissionRoute and feature page"]
  Route --> Hook["Feature Query or mutation hook"]
  Hook --> Service["Feature service"]
  Service --> Axios["Shared Axios instance"]
  Axios --> API["MediCard API"]
```

The diagram shows the usual protected feature path. `/login` and `/change-password` are outside the dashboard layout; some components call services directly for lookups. See routing and authentication.

Source layout

Path	Responsibility
`src/app/` [src/app/App.jsx]	Provider composition, router, Query client
`src/features/` [src/features/providers/pages/ProvidersPage.jsx]	Feature-owned pages, components, hooks, schemas, services, and utilities
`src/shared/api/` [src/shared/api/axiosInstance.js]	Axios instance, token storage functions, interceptors
`src/shared/components/` [src/shared/components/QueryErrorState.jsx]	Reusable fields, modals, selects, tables, pagination, guards, states
`src/shared/hooks/` [src/shared/hooks/useServerPagination.js]	Pagination, debounce, media query, and filter panel hooks

Path	Responsibility
`src/shared/utils/` [src/shared/utils/handleMutationError.js]	Error mapping, JWT, image compression, input, and dashboard filter helpers
`src/layout/` [src/layout/DashboardLayout.jsx], `src/navbar/` [src/navbar/Navbar.jsx], `src/sidebar/` [src/sidebar/Sidebar.jsx]	Dashboard shell and permission-filtered navigation
`src/assets/` [src/assets/Logo.svg], `public/` [public/favicon.svg]	Imported and public static assets

Feature folders generally use `pages/`, `components/`, `hooks/`, `schema/`, and `services/`. Names and capitalization are not fully uniform: examples include `ServiceCategory`, `Medicines`, `categoreis`, `cardActivtion`, and singular `service/` directories. Use existing paths when navigating or importing; do not infer a path from a normalized feature name.

State and data flow

- `AuthProvider` owns the in-memory token, user, permissions, initialization flags, and password-change state. Token values and the password-change flag are stored in `localStorage`; permission codes are not persisted. See authentication.
- TanStack Query owns server data. Feature hooks define query keys and response projections; mutation hooks typically invalidate affected query keys. `useProvidersQuery` [src/features/providers/hooks/useProvidersQuery.js] and `useTicketMutations` [src/features/tickets/hooks/useTicketMutations.js] are representative examples.
- Pages and forms use local React state for open dialogs, draft filters, search terms, and editing state. `useServerPagination` [src/shared/hooks/useServerPagination.js] manages page number and size and resets the page when its caller's reset key changes.
- Dashboard summary and chart filters use `sessionStorage` through `dashboardFilterSession.js`; login, logout, and session invalidation clear them.
- Shared field and form utilities include `TextField` [src/shared/components/TextField.jsx], `ImageUploadField` [src/shared/components/ImageUploadField.jsx], `applyServerErrors` [src/shared/utils/applyServerErrors.js], and `handleMutationError` [src/shared/utils/handleMutationError.js]. A shared component's existence does not mean every feature uses it.

Styling and major libraries

Vite configuration [vite.config.js] loads the React and Tailwind CSS v4 plugins. `src/index.css` holds global styles. React Router handles client navigation, React Hook Form and Zod handle many forms, Recharts renders dashboard charts, and Leaflet/React Leaflet render the map. The exact dependency versions and scripts are in `package.json`.

The architecture is a description of source code, not a guarantee that every backend call or UI flow succeeds in a deployed environment.

CHAPTER 03

Development setup and deployment

README · Architecture · API integration

Requirements and commands

`package.json` declares Node.js `22.x` and uses npm with a committed lockfile [package-lock.json].

BASH

```
npm ci
npm run dev
```

Vite normally serves the local app at `http://localhost:5173` unless the port is occupied or overridden. An API environment and an admin account are needed to use authenticated features; this repository does not provide those credentials.

Command	Checked-in script
`npm run dev`	`vite`
`npm run build`	`vite build`
`npm run preview`	`vite preview`
`npm run lint`	`eslint .`

There is no `test` script or checked-in test suite. The build output is `dist/`, which `eslint.config.js` ignores and `.gitignore` excludes from Git. The repository contains a Prettier configuration [.prettierrc], but no formatting script is declared.

Environment configuration

`src/shared/api/axiosInstance.js` sets the API base URL directly in source. A source search found no project-specific `import.meta.env` variable or checked-in `.env` file. `import.meta.env.DEV` is used only for optional protected-route diagnostics in `Protectedroute.jsx`. Therefore, there are **no required environment variable names to set for the current code**. Changing the API target would require a code or deployment change outside the scope of this documentation.

The front end stores access and refresh tokens in browser `localStorage`; never put real credentials or token values in documentation, commits, or screenshots. Authentication and permissions describes the lifecycle.

Build and hosting

`npm run build` creates static assets in `dist/`. `npm run preview` serves that build locally for inspection. `vercel.json` rewrites paths to `/index.html` so direct navigation to React Router URLs works on Vercel. Another static host needs an equivalent SPA fallback. The repository does not define a CI pipeline, deployment environment variables, release process, or server configuration beyond this rewrite; obtain those details from the deployment owners.

The lint configuration [eslint.config.js] applies recommended JavaScript, React Hooks, and Vite React Refresh rules to `.js` and `.jsx` files. Documentation-only changes do not require a build to establish their contents; a build is a separate application validation step.

CHAPTER 04

Routing and layouts

README · Authentication

The router [src/app/router.jsx] uses `createBrowserRouter`. Most page components are loaded with React `lazy` and rendered beneath the `App` Suspense boundary [src/app/App.jsx]. The notifications page is imported directly. `DashboardLayout` [src/layout/DashboardLayout.jsx] supplies the sidebar, navbar, and an `Outlet` for protected pages.

Route boundaries

- `/login` is public. The login page redirects an already authenticated user to `/` or `/change-password`.
- `/change-password` is outside the dashboard layout. Its page redirects unauthenticated users to `/login` and users without the password-change flag to `/`.
- The `/` route wraps dashboard children in `ProtectedRoute` [src/shared/components/Protectedroute.jsx]. It waits for authentication and permission loading, redirects an invalid session to `/login`, renders a retryable permission-loading error, and redirects a user flagged for password change.
- Most dashboard pages are wrapped in `PermissionRoute` [src/features/auth/components/PermissionRoute.jsx], which renders `AccessDeniedPage` [src/shared/components/AccessDeniedPage.jsx] when the required code is absent. The nested `*` route renders a not-found page.
- Sidebar navigation [src/sidebar/navigation.js] separately filters links by permission. Hiding a link does not replace route checks or backend authorization.

Route inventory

The table describes the permission wrapper in source, not a claim about server-side access. The authoritative code strings are in `permissionCodes.js`.

Path	Page	Route permission
/	Dashboard	`dashboard.read`
/providers	Providers	`providers.read`
/providers/new	Provider creation	`providers.create`
/providers/:id	Provider details	`providers.read`
/branches	Branches	`provider_branches.read`
/provider-map	Provider map	`map.read`
/provider-category	Provider categories	`provider_categories.read`

Path	Page	Route permission
`/service-category`	Service categories	`service_categories.read`
`/services`	Services	`services.read`
`/medicines`	Medicines	`medicines.read`
`/speialist`	Specialists	`specialists.read`
`/governorates`	Governorates	`governorates.read`
`/cities`	Cities	`cities.read`
`/clients`	Clients	`clients.read`
`/clients/:clientId`	Client details	`clients.read`
`/invoices`	Invoices	`invoices.read`
`/support-tickets`	Tickets	`tickets.read`
`/ticket-types`	Ticket types	`ticket_types.read`
`/plan-types`	Subscription types	`plan_types.read`
`/card-activation`	Card activation listing	`cards.read`
`/card-pools`	Card pools	`card_pools.read`
`/card-pool/sold-cards`	Sold cards in a pool	`card_sales.read`
`/sold-card`	Sold card records	`card_sales.read`
`/card-missed`	Missed card records	`missed_cards.read`
`/offer-sliders`	Offer sliders	`sliders.read`
`/offer-sliders/:id`	Slider details	`sliders.read`
`/employees`	Employees	`employees.read`
`/notifications`	Marketing notifications	`notification_marketing.read`
`/terms-and-conditions`	Terms and conditions	`terms_and_conditions.read`
`/refund-policy`	Refund policy	`refund_policy.read`
`/privacy-policy`	Privacy policy	`privacy_policy.read`
`/contact-us`	Contact information	`contact_us.read`

`/speialist` is the route's current spelling. Provider branches, services, and packages also appear as permission-filtered tabs within provider details; their tab behavior is described in providers and map. The sidebar is a subset of the route inventory, so a valid route need not have its own sidebar entry.

CHAPTER 05

Authentication and permissions

README · Routing · API integration

This describes client behavior. The repository does not establish the backend's token validity rules, refresh-token policy, or server-side authorization enforcement.

Login and stored session

`useAuthMutations` [src/features/auth/hooks/useAuthMutations.js] sends `POST AuthAdmin/login` with `email` and `password`, mapping the login form's `identifier` field to `email`. The success handler reads a token from `data.token` or `data.accessToken`, an optional `data.refreshToken`, `data.isNeedChangePassword`, and permission codes from `data.permissionCodes` with `data.permissions` as a fallback. It accepts `succeeded` or `success` on the outer response. The exact live response shape is backend-dependent.

`AuthProvider` [src/features/auth/context/Authcontext.jsx] stores the access token under `token`, refresh token under `refreshToken`, and password-change flag under `mustChangePassword` in browser `localStorage`. It decodes JWT claims [src/shared/utils/jwt.js] such as `employeeId` and `accountType` to form a user object. Decoding is a client-side convenience, not proof that the server accepts a token. Login clears old dashboard filters and legacy persisted permission keys.

The login page uses React Hook Form and the login Zod schema [src/features/auth/schema/loginSchema.js]. It sends the validated form through the mutation hook. Credentials are never documented as values here.

Startup, request, and refresh flow

MERMAID DIAGRAM SOURCE

```
flowchart TD
  Start["AuthProvider starts"] --> Stored{"Stored token has admin-identifying claim?"}
  Stored -- No --> Clear["Clear local session"]
  Stored -- Yes --> Expired{"JWT expired?"}
  Expired -- Yes --> Refresh["POST AuthAdmin/refresh-token"]
  Expired -- No --> Permissions["Load employee and permission catalog"]
  Refresh -- Success --> Permissions
  Refresh -- Failure --> Clear
  Permissions -- Success --> Route["ProtectedRoute may render children"]
  Permissions -- Failure --> Retry["ProtectedRoute shows retryable error"]
  Route --> Request["Feature request with Bearer token"]
  Request --> Unauthorized{"HTTP 401?"}
  Unauthorized -- Yes --> RefreshRetry["Refresh once and retry request"]
  RefreshRetry -- Failure --> Clear
```

The Axios request interceptor [src/shared/api/axiosInstance.js] adds `Authorization: Bearer <token>` when a token exists. It rejects normal requests without a token and blocks most requests while a forced password change is required. Login, refresh, and logout are treated as auth requests; change-password and logout are allowed during the password-change state.

On startup, `AuthProvider` checks a stored JWT and refreshes it only if it is already expired. The response interceptor refreshes after an unauthorized feature request. A single in-flight refresh promise is shared, a request sent with an older token may retry with the newer stored token, and refresh attempts are limited to three within ten seconds. A second 401 after retry or refresh failure clears tokens and signals session expiry. There is **no proactive timer-based refresh** in the current source. The refresh call reads `response.data.data.token` and, if present, `response.data.data.refreshToken`.

`ProtectedRoute` [src/shared/components/Protectedroute.jsx] waits for initialization, checks authentication and JWT expiration, clears Query data on invalid state, redirects to `/login`, and sends users with `mustChangePassword` to `/change-password`. On permission-loading failure it preserves the authenticated session and presents a retry action. The standalone `SessionExpiredModal` [src/features/auth/components/SessionExpiredModal.jsx] exists in source but is not mounted by `App` or the route tree; do not describe it as the displayed expiry flow.

Password change and logout

The forced password page [src/features/auth/pages/ForceChangePasswordPage.jsx] hosts `ChangePasswordModal` [src/features/auth/components/ChangePasswordModal.jsx]. That modal uses local form state and the Zod password schema. The mutation sends `currentPassword`, `newPassword`, and `confirmNewPassword` to `POST AuthAdmin/change-password`. On a successful response, the client clears its session and returns to login. The user menu also opens the modal for a voluntary password change.

Logout sends `POST AuthAdmin/logout`, then clears the local session and Query cache and redirects to login whether the request succeeds or fails. This is client behavior; backend token revocation is unverified.

Permission resolution and use

The client does not persist permission codes. At login it normalizes codes supplied in the login response. On startup, `AuthProvider` [src/features/auth/context/Authcontext.jsx] fetches `GET EmployeeAdmin/{employeeId}` and `GET PermissionAdmin`, maps the employee's `permissionIds` through catalog entries with `id` and `code`, and places the resulting codes in context. Missing IDs, unresolved IDs, or a failed request produce a permission initialization error instead of granting access. Editing the current employee's permissions triggers a reload through `useEmployeePermissionsMutation` [src/features/employees/permissions/hooks/useEmployeePermissionsMutation.js].

`permissionCodes.js` contains the strings used by the UI. `hasPermission` checks exact code membership for an authenticated user. `PermissionRoute` [src/features/auth/components/PermissionRoute.jsx] guards pages; `PermissionGuard` [src/features/auth/components/PermissionGuard.jsx], sidebar navigation [src/sidebar/navigation.js], and feature components hide or disable specific actions. The provider details page and package tab provide examples of nested action checks. These checks describe the front-end UI only; whether the backend rejects unauthorized calls must be verified with backend owners or integration tests.

CHAPTER 06

API integration and client request inventory

README · Authentication · Forms and UI states

Scope and HTTP client

This is an inventory of requests **verified in front-end source**, not a validated OpenAPI specification. Paths, methods, parameters, and fields below describe what the browser code sends or reads. Live backend acceptance, complete response schemas, side effects, and authorization need backend or runtime verification.

`axiosInstance.js` creates one Axios instance with base URL configured in `src/shared/api/axiosInstance.js` and default `Content-Type: application/json`. Feature services import it. A request interceptor attaches a stored bearer token and blocks certain unauthenticated or forced-password-change requests. A response interceptor handles HTTP 401 refresh and retry; it does not unwrap every response or globally reject `succeeded: false`. Authentication gives the full session flow.

Services usually receive Axios `response.data`, which may be an application envelope. They differ in what they return:

Observed client expectation	Examples
Return the envelope and let a hook read `data.items`	Providers [src/features/providers/services/providersService.js] and provider packages [src/features/providerPackages/services/providerPackagesService.js] return the response body; their hooks read nested `data.items`.
Unwrap `response.data.data` in the service	Employees [src/features/employees/services/employeesService.js], card pools [src/features/cardPool/services/cardPoolService.js], and invoices [src/features/invoices/services/invoicesService.js] return inner data for lists.
Require an application success flag	Permission catalog [src/features/employees/permissions/services/permissionsService.js], map [src/features/provider-map/services/ProviderMapService.js], and some update services throw when `succeeded` is false.
Assume a different or uncertain shape	Branch hooks [src/features/branches/hooks/useBranchesQuery.js] read top-level `items` from a service that returns the response body unchanged. The live shape is unverified.

Where list data is read as a paginated object, the observed fields are `items`, `pageNumber`, `pageSize`, `totalCount`, and `totalPages`. This is a **front-end expectation**, not a guarantee for all endpoints. `useClientsQuery` [src/features/clients/hooks/useClientsQuery.js] recalculates total pages from count and page size. `useServerPagination` [src/shared/hooks/useServerPagination.js] starts at page 1 with size 20 unless overridden, resets page on a reset key, and leaves filter construction to each feature. Several list hooks use TanStack Query `placeholderData` to retain previous data during pagination. Query keys, invalidation, and enabled conditions are owned by each feature hook; the root Query client [src/app/queryClient.js] has no custom defaults.

Other response fields explicitly read by the client include login `token` / `accessToken`, `refreshToken`, `isNeedChangePassword`, and `permissionCodes` / `permissions`; employee `permissionIds`; permission catalog `id` and `code`; map `providers` and `governorates`; ticket detail `ticketTimelines`; and provider package `packageId`, names, prices, discount, service count, and active state. These are partial client projections, not complete response schemas.

Authentication and staff

Sources: auth service [src/features/auth/services/Authservice.js], Axios refresh [src/shared/api/axiosInstance.js], employee service [src/features/employees/services/employeesService.js], permission service [src/features/employees/permissions/services/permissionsService.js].

Method	Path relative to `/api/`	Parameters or body sent by client
POST	`AuthAdmin/login`	`email`, `password`
POST	`AuthAdmin/refresh-token`	`refreshToken`; sent with base Axios, outside the shared interceptors
POST	`AuthAdmin/change-password`	`currentPassword`, `newPassword`, `confirmNewPassword`
POST	`AuthAdmin/logout`	No explicit body
POST	`/AuthAdmin/register`	`firstName`, `lastName`, `email`, `phoneNumber`, numeric `employeeMemberId`
PATCH	`AuthAdmin/employees/{employeeId}/reset-password`	No explicit body
GET	`/EmployeeAdmin`	`PageNumber`, `PageSize`, optional `SearchTerm`, `EmployeeGroupId`
GET	`/EmployeeAdmin/{employeeId}`	Path ID
PUT	`/EmployeeAdmin/{employeeId}`	`firstName`, `lastName`, `email`, `phoneNumber`, numeric `employeeGroupId`
PATCH	`/EmployeeAdmin/{employeeId}/toggle-active`	No explicit body
GET	`PermissionAdmin`	No explicit parameters; client expects catalog entries with `id` and `code`
PUT	`PermissionAdmin/employees/{employeeId}`	`permissionIds` array
GET	`/AdminLookup/employee-groups`	`SearchTerm`, `PageNumber`, optional `PageSize`

Login reads an outer `succeeded` or `success` and nested or fallback token fields as described in authentication. The employee list service returns inner `data` after checking `succeeded`; registration checks `success`. Those differences are retained in the inventory because the code does not establish one uniform response contract.

Providers, branches, services, and packages

Sources: providers [src/features/providers/services/providersService.js], branches [src/features/branches/services/providerBranchesService.js], provider services [src/features/providerServices/services/providerServicesService.js], packages [src/features/providerPackages/services/providerPackagesService.js].

Method	Path relative to `/api/`	Parameters or body sent by client
GET	`/admin/providers`	`PageNumber`, `PageSize`, `SearchTerm`, `Search`, `Status`, `FromDate`, `ToDate`, `CategoryId`, `SpecialistId`
GET	`/admin/providers/{id}`	Path ID
POST, PUT	`/admin/providers`; `/admin/providers/{id}`	Multipart `ProviderCategoryId`, optional `SpecialistId`, `ArName`, `EnName`, `HotLine`, `PhoneNumber`, `IsActive`, `Landline`, optional `Logo`
DELETE	`/admin/providers/{id}`	Path ID
PATCH	`/admin/providers/make-active/{id}`	Path ID
GET	`/AdminLookup/provider`	`PageNumber`, `PageSize`, `SearchTrem` (source spelling)
GET	`/ProviderBranchesAdmin`	Optional `ProviderId`, `GovernorateId`, `CityId`, `Search`, `Status`, `PageNumber`, `PageSize`
GET	`AdminLookup/provider-branches`	Optional `ProviderId`, `SearchTerm`, `PageNumber`, `PageSize`
POST, PUT	`/ProviderBranchesAdmin/provider-branches`; `/ProviderBranchesAdmin/provider-branches/{id}`	JSON `providerId`, `governorateId`, `cityId`, `email`, `branchName`, `mapUrl`, `fullAddress`, `isActive`
DELETE, PATCH	`/ProviderBranchesAdmin/{id}`; `/ProviderBranchesAdmin/make-active/{id}`	Deactivate and activate requests respectively
GET	`/ProviderServiceAdmin/provider-services/{providerId}`	`PageNumber`, `PageSize`, `SearchTerm`, `Status`
POST, PUT	`/ProviderServiceAdmin/provider-services`; `/ProviderServiceAdmin/provider-services/{id}`	JSON `providerId`, `serviceId`, `priceBefore`, `discountPercentage`, `isSpecialOffer`, `isActive`
PATCH, DELETE	`/ProviderServiceAdmin/provider-services/{id}/activate`; `/ProviderServiceAdmin/provider-services/{id}`	Activate and deactivate requests respectively
GET	`/ProviderPackageAdmin`	`ProviderId`, `PageNumber`, `PageSize`

Method	Path relative to `/api/`	Parameters or body sent by client
GET	`/ProviderPackageAdmin/{packageId}`	Path ID; service reads `data.data`
POST, PUT	`/ProviderPackageAdmin`	Multipart `ProviderId`, `NameAr`, `NameEn`, `ImageAr`, `ImageEn`, `DescriptionAr`, `DescriptionEn`, `PriceBefore`, `DiscountPercentage`, repeated `ServiceIds`; PUT also sends `PackageId` and omits unchanged image files
PATCH, DELETE	`/ProviderPackageAdmin/{packageId}/toggle-active`; `/ProviderPackageAdmin/{packageId}`	Status toggle and delete requests

The provider service writes image and text fields as `FormData`. Package create also uses `FormData`, while package update includes an image only if the form value is a new `File`. The client displays package `priceAfter` and `totalDiscount` from responses; it does not establish the backend price formula.

Clients, invoices, tickets, and dashboard

Sources: clients [src/features/clients/services/clientsService.js], invoices [src/features/invoices/services/invoicesService.js], invoice hook [src/features/invoices/hooks/useInvoicesQuery.js], tickets [src/features/tickets/services/ticketsService.js], dashboard [src/features/dashboard/services/dashboardService.js].

Method	Path relative to `/api/`	Parameters or body sent by client
GET	`/ClientAdmin`	List hook sends `searchTerm`, `platformType`, `isMale`, `hasCard`, optional pool `id` and `sourceType`, `pageNumber`, `pageSize`
GET	`/ClientAdmin/export`	Export filters, `responseType: blob`
PUT	`/ClientAdmin/block`	`userId`, `isBlocked`; source calls the method an assumption
GET	`/ClientAdmin/{id}`	Path ID
GET	`/ClientAdmin/order-history`	`UserId`, `ProviderId`, `Status`, UTC `FromDate` and `ToDate`, `PageNumber`, `PageSize`
GET	`/ClientAdmin/{clientId}/family-members`	Path ID
GET	`/ProviderBranchView/invoices/{invoiceId}/details`	Path ID, used from client details
GET	`/AdminLookup/clients`	`SearchTerm`, `PageNumber`, `PageSize`

Method	Path relative to `/api/`	Parameters or body sent by client
GET	`/admin/invoices`	`PageNumber`, `PageSize`, `SearchTerm`, `ClientId`, `ProviderId`, `ProviderBranchId`, `Status`, UTC `FromDate`/`ToDate`, `IsCash`
GET	`/admin/invoices/export`	Similar filters without page fields, `responseType: blob`
GET	`/ApprovalHistory/invoices/{orderNo}/timeline`	Path order number; request accepts an abort signal
GET	`/admin/tickets`	`SearchTerm`, `ClientId`, `TicketTypeId`, `ProviderId`, `EmployeeGroupId`, `Priority`, `IsClosed`, `PageNumber`, `PageSize`
GET	`/admin/tickets/{id}`	Path ID
POST	`/admin/tickets`	`ticketTypeId`, `userId`, nullable `providerId` and `assignedToGroupId`, `userPhoneNumber`, `priority`, `description`
PUT	`/admin/tickets/{id}`	Nonempty fields of a feature-built update payload, including status/closed state and optional reply
GET	`admin/dashboard`	`FromDate`, `ToDate`
GET	`admin/dashboard/chart`	`GroupBy`, `Year`

Client order-history and invoice date filters are converted to UTC start/end-of-day strings in their respective hooks/services. Invoice and client exports are Blob requests. The timeline service checks `succeeded` before returning inner data. Ticket create/update return the response body unchanged; the details UI reads nested `data` and `ticketTimelines`.

Cards and location map

Sources: card pools [src/features/cardPool/services/cardPoolService.js], sold cards [src/features/sold-card/services/cardSoldService.js], missed cards [src/features/missedCard/services/cardMissedService.js], card activation [src/features/cardActivtion/services/cardActivationService.js], provider map [src/features/provider-map/services/ProviderMapService.js], client map [src/features/provider-map/services/ClientMapService.js].

Method	Path relative to `/api/`	Parameters or body sent by client
GET	`/CardPoolAdmin`	`SearchTerm`, UTC `FromDate`/`ToDate`, `PageNumber`, `PageSize`
GET	`/CardPoolAdmin/sold-cards`	`CardPoolId`, `SearchTerm`, `PageNumber`, `PageSize`
POST	`/CardPoolAdmin`	URL-encoded `count`

Method	Path relative to `/api/`	Parameters or body sent by client
GET, DELETE	`/CardPoolAdmin/{id}/export`; `/CardPoolAdmin/{id}`	Export uses `responseType: blob`
GET	`/admin/card-solds`	`SearchTerm`, `FromDate`, `ToDate`, `PageNumber`, `PageSize`
POST	`/admin/card-solds/by-count`	Multipart `Count`, `ClientName`, `ClientPhone`, `ProofPayment`
POST	`/admin/card-solds/by-numbers`	Multipart repeated `CardNumbers`, `ClientName`, `ClientPhone`, `ProofPayment`
GET, DELETE	`/admin/card-solds/{id}/export`; `/admin/card-solds/{id}`	Export uses `responseType: blob`
GET	`/CardMissedAdmin`	`CardPoolId`, `SearchTerm`, `FromDate`, `ToDate`, `MissingType`, `PageNumber`, `PageSize`
POST, DELETE	`/CardMissedAdmin`; `/CardMissedAdmin/{id}`	Create sends `cardNumber` and `missingType`
GET	`/CardAdmin/not-activated`	`Id`, `SourceType`, `SearchTerm`, `PageNumber`, `PageSize`
GET	`/CardAdmin/not-activated-cards/export`	Same optional filters, `responseType: blob`
GET	`/MapAdmin/data`	Optional `providerCategoryId`; service expects `data.providers` and `data.governorates`
GET	`/MapAdmin/clients`	Service expects a location array in `data`

The card activation service contains no activation mutation. Map hooks validate and aggregate received coordinates locally, as described in providers and map.

Catalogs, notifications, and content

Sources: provider categories [src/features/categoreis/service/categoryService.js], service categories [src/features/ServiceCategory/services/Servicecategoriesservice.js], services [src/features/services-admin/services/ServicesService.js], medicines [src/features/Medicines/services/Medicinesservice.js], specialists [src/features/specialists/service/SpecialistsService.js], governorates [src/features/governorates/services/governoratesService.js], cities [src/features/cities/service/citeisService.js], ticket types [src/features/ticket-types/services/ticketTypesService.js], plan types [src/features/subscription-types/services/subscriptionTypesService.js], sliders [src/features/offer-sliders/service/sliderService.js], notifications [src/features/notificationsAdmin/services/notificationsService.js].

Resource and paths relative to `/api/`	Methods and request details
`/ProviderCategoryAdmin/providerCategories`; `/ProviderCategoryAdmin/categories/{id}`	GET list with `PageNumber`, `PageSize`, `SearchTerm`; POST multipart bilingual names and `Logo`; PUT multipart names, `IsUpdatedImage`, optional `ImageUrl`; DELETE by ID

Resource and paths relative to `/api/`	Methods and request details
`/ServiceCategoryAdmin/ServiceCategories`; `/ServiceCategoryAdmin/categories/{id}`	GET list with `PageNumber`, `PageSize`, `SearchTerm`; POST and PUT multipart bilingual names/image fields; DELETE by ID
`/ServiceAdmin/services`; `/ServiceAdmin/services/{id}`	GET list/detail; list uses `PageNumber`, `PageSize`, `SearchTerm`, `CategoryId`, `Status`; POST/PUT JSON category, bilingual names, optional `serviceInstruction` and `cpt`; PUT also sends `isActive`; DELETE by ID
`/MedicineAdmin/medicines`; `/MedicineAdmin/medicines/{id}`	GET list/detail; list uses `PageNumber`, `PageSize`, `SearchTerm`; POST/PUT multipart names, price, form, optional image; PUT includes `isUpdatedImage`; DELETE by ID
`/SpecialistAdmin`; `/SpecialistAdmin/{id}`	GET list with page and search; POST/PUT bilingual names; DELETE by ID
`/GovernoratesAdmin`; `/GovernoratesAdmin/{id}`	GET list with page and search; POST/PUT bilingual names; DELETE by ID
`/CityAdmin/cities`; `/CityAdmin/cities/{id}`	GET list with page, search, `GovernorateId`, `IsActive`; POST/PUT bilingual names and `governorateId`; PUT also sends `cityId`; DELETE by ID
`/ticket-types`; `/ticket-types/{id}`	GET list with `Search` and page fields; POST/PUT bilingual names; DELETE by ID
`/PlanTypeAdmin`; `/PlanTypeAdmin/{id}`; `/PlanTypeAdmin/{id}/toggle-active`	GET list with search and page fields; POST/PUT bilingual name/description, `priceBefore`, `discountPercentage`; DELETE and PATCH status toggle
`/SliderAdmin`; `/SliderAdmin/{id}`	GET list/detail; list uses `PageNumber`, `PageSize`, `ProviderName`; POST/PUT multipart `ProviderId` and English/Arabic images; PUT includes `Id` and image-change flags; DELETE by ID
`admin/notification-marketing`; `admin/notification-marketing/{notificationId}`	GET list, POST notification payload, DELETE by ID
`/AdminLookup/packages`	GET with no query parameters; client filters and paginates the returned array locally

The notification schema [src/features/notificationsAdmin/schema/notificationSchema.js] maps form values to POST fields `title`, `messageEn`, `messageAr`, nullable `imageUrl`, `isMale`, `governmentId`, `targetId`, `durationDay`, numeric `targetType`, and nullable `scheduledAt`. Backend delivery and scheduling semantics are unverified.

Content services use the following calls: terms [src/features/termsAndConditions/services/Termsandconditionsservice.js] `GET/POST /TermsAndConditionsAdmin`, refund policy [src/features/refundPolicy/services/RefundPolicyservice.js] `GET/POST /RefundPolicyAdmin`, privacy policy [src/features/privacyPolicy/services/PrivacyPolicyservice.js] `GET/POST /PrivacyPolicyAdmin`, and contact information [src/features/contactUs/services/contactsUsService] `GET/POST /admin/contact-us`. Policy POST bodies contain `descriptionEn` and `descriptionAr`. Contact POST sends `phoneNumber`, `email`, `whatsApp`, and nullable `faceBook`, `instagram`, `tikTok`.

Unverified contract points

- `PUT /ClientAdmin/block` is explicitly identified as an assumption in its service.
- Branch list hooks expect top-level pagination fields, while the service returns the response body unchanged. The backend shape has not been observed in this audit.
- `SearchTrem` in provider lookup is the source spelling; whether the server recognizes it is unknown.
- Application envelopes and `succeeded` / `success` flags are not handled uniformly by all services. A successful HTTP status does not necessarily mean every feature checks the application flag.
- Export file contents, permission enforcement, mutation side effects, notification delivery/timezone, and exact error payloads need backend or runtime verification.

CHAPTER 07

Forms and UI states

README · API integration

Form patterns

Many feature forms use React Hook Form with `zodResolver` and feature-owned Zod schemas. Examples are login [src/features/auth/pages/LoginPage.jsx], provider packages [src/features/providerPackages/components/ProviderPackageFormModal.jsx], tickets [src/features/tickets/components/TicketFormModal.jsx], and notifications [src/features/notificationsAdmin/components/NotificationFormModal.jsx]. The forced password modal [src/features/auth/components/ChangePasswordModal.jsx] instead uses local React state with schema validation. Do not assume all forms use one implementation.

Schemas express client-side requirements only. For example, provider package validation [src/features/providerPackages/schema/providerPackageSchema.js] requires bilingual names and descriptions, a nonnegative price, a 0–100 discount percentage, images, and at least one selected service. Create requires image files; update accepts existing image strings or files. The form converts numeric strings and selected service objects before the service constructs multipart data. The backend may impose additional requirements.

Shared controls include `TextField` [src/shared/components/TextField.jsx], `NumberInput` [src/shared/components/NumberInput.jsx], `ImageUploadField` [src/shared/components/ImageUploadField.jsx], `SearchableAsyncSelect` [src/shared/components/SearchableAsyncSelect.jsx], `FormModalShell` [src/shared/components/FormModalShell.jsx], and `ConfirmDeleteModal` [src/shared/components/ConfirmDeleteModal.jsx]. Image selection can use client-side compression [src/shared/utils/compressImage.js]. Bilingual fields and the rich-text editor preserve Arabic text direction in their own UI.

Server validation and mutation failures

`handleMutationError` [src/shared/utils/handleMutationError.js] distinguishes connection errors, HTTP 5xx, 401, 403, an `errors` array, an `errors` object, and a general backend message. It shows toast messages for most cases and passes an error object to the feature when field mapping is possible. `applyServerErrors` [src/shared/utils/applyServerErrors.js] lowercases the first character of each backend field key, chooses the first message in an array, humanizes it, and calls React Hook Form's `setError`. It does not normalize arbitrary nested paths or guarantee a matching form control.

Not every feature uses that shared flow. Notification mutations [src/features/notificationsAdmin/hooks/useNotificationMutations.js] flatten backend errors into a local page toast. ProviderInfoForm [src/features/providers/components/ProviderInfoForm.jsx] applies backend errors directly. Ticket details utilities [src/features/tickets/utils/ticketDetailsUtils.js] map update errors to known ticket fields. A non-HTTP error without `error.response` is commonly described as a connection failure by the shared helper; that message is the helper's behavior, not proof of the underlying cause.

Loading, empty, success, and error states

Situation	Implemented examples
Route loading	`App` Suspense [src/app/App.jsx] renders a spinner while a lazy page loads; `ProtectedRoute` [src/shared/components/Protectedroute.jsx] renders a session-loading spinner.
Query loading	Provider packages [src/features/providerPackages/components/ProviderPackagesTab.jsx] show card skeletons; dashboard cards [src/features/dashboard/components/StatCards.jsx] show skeletons. Other pages use table loading indicators.
Empty data	`TableEmptyState` [src/shared/components/TableEmptyState.jsx] can distinguish an empty list from a filtered search; provider packages [src/features/providerPackages/components/ProviderPackagesTab.jsx] use it for an empty result.
Query failure	`QueryErrorState` [src/shared/components/QueryErrorState.jsx] distinguishes a response error from a request with no response and exposes a retry button. Features can instead render their own error UI.
Successful mutation	Many mutation hooks show a React Hot Toast success message, close a modal, and invalidate Query data; ticket mutations [src/features/tickets/hooks/useTicketMutations.js] are one example.

Coverage varies by page. For example, `InvoicesPage` [src/features/invoices/pages/InvoicesPage.jsx] reads `data` and `isLoading` from its query but does not render its `isError` or `error` state. The docs describe this as current behavior, not as an intended error policy. The shared Axios response interceptor handles only HTTP 401; it does not globally turn application-level `succeeded: false` responses into exceptions. Individual services or hooks sometimes check that flag.

Exports use a separate loading flag and a browser Blob download; see clients and operations.

CHAPTER 08

Providers and map

README · API integration · Permissions

Provider directory and details

`ProvidersPage` [src/features/providers/pages/ProvidersPage.jsx] lists providers with server pagination, debounced search, status/date/category/specialist filters, and create or status actions gated by permission. `providersService.js` sends those parameters to `GET /admin/providers`; create and update send multipart form data with bilingual names, category, optional specialist, contact fields, active state, and an optional logo. The provider lookup sends `SearchTrem` as spelled in source. Whether that spelling is accepted by the live API is not verified here.

`ProviderDetailsPage` [src/features/providers/pages/ProviderDetailsPage.jsx] is shared by `/providers/new` and `/providers/:id`. It can present the main provider form and, for an existing provider, permission-filtered branches, services, and packages tabs. An edit request also checks `providers.update`. Route-level provider read permission alone does not expose every nested tab or action.

Branches and provider services

The standalone branches page [src/features/branches/pages/BranchesPage.jsx] renders the branch tab in read-only mode. The tab and its service [src/features/branches/services/providerBranchesService.js] support provider-specific or all-provider lists, geographic filters, search, status, pagination, create/update, and activate/deactivate actions. The branch UI uses status values 0, 1, and 2 for all, active, and deactive. The service returns the response body unchanged while the hooks [src/features/branches/hooks/useAllBranchesQuery.js] read top-level `items` and pagination fields; the actual response shape must be checked against the backend before assuming the list works.

Provider services [src/features/providerServices/components/ProviderServicesTab.jsx] associate a provider with a catalog service and price, discount, special-offer, and active fields. `providerServicesService.js` uses a provider-scoped list, JSON create/update, a PATCH activation call, and a DELETE deactivation call. These are client request choices; backend effects are unverified.

Packages

`ProviderPackagesTab` [src/features/providerPackages/components/ProviderPackagesTab.jsx] lists paginated packages, displays price and discount information received from the API, and permission-gates create, edit, status, and delete controls. The form [src/features/providerPackages/components/ProviderPackageFormModal.jsx] validates bilingual content, percentage and price, two images, and selected provider services, then converts values for multipart create/update requests [src/features/providerPackages/services/providerPackagesService.js]. On update, image fields are sent only when a new `File` is supplied. The detail request reads `data.data`; the list hook reads `data.data.items`. Successful mutations invalidate package Query keys. The actual backend price calculation and deletion semantics are not established by the front end.

Map

`ProviderMapPage` [src/features/provider-map/pages/ProviderMapPage.jsx] fetches provider/branch map data by provider category and can fetch client locations when the “Users” toggle is on. `useProvidersMapQuery` [src/features/provider-map/hooks/useProvidersMapQuery.js] filters invalid Egypt coordinates, resolves governorates from local polygon data, and applies governorate and search filtering on the client. It builds branch markers and governorate/country bubbles. `usecCientLocationsQuery` [src/features/provider-map/hooks/usecCientLocationsQuery.js] groups valid client coordinates into fixed 200-meter grid cells and aggregate bubbles. `ProviderClusterMap` [src/features/provider-map/components/ProviderClusterMap.jsx] renders the Leaflet view. The presence or accuracy of live location data is backend-dependent.

CHAPTER 09

Clients and operations

README · API integration · Forms and UI states

Dashboard, clients, and invoices

`DashboardPage` [src/features/dashboard/pages/DashboardPage.jsx] combines summary cards and charts. The dashboard service [src/features/dashboard/services/dashboardService.js] sends date-range parameters to `GET admin/dashboard` and grouping/year parameters to `GET admin/dashboard/chart`. Summary and chart filters are saved separately in browser `sessionStorage` by `dashboardFilterSession.js`, and session changes clear them. The grouping codes in source are month 1, quarter 2, half-year 3, and year 4; server aggregation rules are unverified.

`useClientsQuery` [src/features/clients/hooks/useClientsQuery.js] builds server list parameters for search, platform type, sex, card possession, optional pool source, and pagination. It computes `totalPages` from `totalCount` and page size because a source comment reports inconsistent backend `totalPages` values. Client details [src/features/clients/pages/ClientDetailsPage.jsx] bring together the client record, family members, order history, and invoice preview/detail flows. `clientsService.js` sends a `PUT /ClientAdmin/block` request for block/unblock, but explicitly labels that method an assumption; the live backend contract remains unverified.

`InvoicesPage` [src/features/invoices/pages/InvoicesPage.jsx] uses debounced search, draft/applied filters, server pagination, an invoice table, detail modal, timeline, and export. `useInvoicesQuery` [src/features/invoices/hooks/useInvoicesQuery.js] converts filter dates to UTC day boundaries and sends client/provider/branch/status/payment filters. `invoicesService.js` reads list data from the response envelope and fetches approval history by order number. The page currently does not display its query error state. Client and invoice exports request Blobs; export hooks [src/features/invoices/hooks/useExportInvoices.js] create downloadable `.xlsx` filenames. The returned file type and contents require backend verification.

Tickets

`TicketsPage` [src/features/tickets/pages/TicketsPage.jsx] shows a filtered, paginated ticket list with create and detail flows. `ticketsService.js` sends search, client, ticket type, provider, employee group, priority, and closed-state filters; create sends ticket metadata and description. Details come from `GET /admin/tickets/{id}`. `TicketDetailsModal` [src/features/tickets/components/TicketDetailsModal.jsx] displays overview and timeline data and prevents editing when `isClosed` is true. `ticketDetailsUtils.js` constructs an update payload and maps known validation fields. The UI's status labels and priority options come from local constants; backend workflow transitions are not established here. Ticket types have their own CRUD screen and service.

Card operations

Card pools [src/features/cardPool/services/cardPoolService.js] support paginated searching and date filtering, pool creation with URL-encoded `count`, sold-card listing by pool, export, and delete. Sold-card records [src/features/sold-card/services/cardSoldService.js] support creation by count or by card numbers with multipart payment proof, list,

export, and delete. Missed-card records [src/features/missedCard/services/cardMissedService.js] support list, create, and delete; source comments label missing type 0 as missed and 1 as damaged. Card activation [src/features/cardActivtion/services/cardActivationService.js] is a listing of non-activated cards and an export request; the source contains no activation mutation. These descriptions refer to UI and HTTP calls, not to confirmed inventory or financial effects on the server.

CHAPTER 10

Staff and notifications

README · Authentication and permissions · API integration

Employees and permission editing

`EmployeesPage` [src/features/employees/pages/EmployeesPage.jsx] lists employees with search, group filtering, and pagination. Its feature hooks support registration, editing, active-state toggle, password reset, and permission editing. `employeesService.js` sends employee list/details and update requests; registration goes to `POST /AuthAdmin/register`, and reset goes to `PATCH AuthAdmin/employees/{employeeId}/reset-password`. The service checks `succeeded` for several operations, but checks `success` for registration. Documented fields and response flags are those read by the client; backend behavior has not been exercised.

`EmployeePermissionsModal` [src/features/employees/permissions/components/EmployeePermissionsModal.jsx] loads an employee's `permissionIds` and the catalog of `id` / `code` entries, groups codes for display, and submits selected IDs. Its UI auto-selects a group's `read` permission when another action is selected and clears other actions when `read` is removed. `useEmployeePermissionsMutation` [src/features/employees/permissions/hooks/useEmployeePermissionsMutation.js] updates list cache, invalidates employee queries, and reloads the signed-in employee's permissions when that employee was edited. The source does not prove that the backend requires the same grouping rule.

Marketing notifications

`NotificationsPage` [src/features/notificationsAdmin/pages/NotificationsPage.jsx] lists marketing notifications and permission-gates create and delete buttons. Its page uses a local four-second success/error toast, while the mutations [src/features/notificationsAdmin/hooks/useNotificationMutations.js] invalidate the notifications query.

`NotificationFormModal` [src/features/notificationsAdmin/components/NotificationFormModal.jsx] uses React Hook Form and `notificationSchema.js`. It validates title, English and Arabic messages, optional URL and targeting fields, an immediate or scheduled send mode, and a future local date/time for scheduling. `toPayload` sends nullable optional fields and a `scheduledAt` string only for the scheduled mode. Target type 1 is labeled Provider and 2 Package in the UI. The notification service [src/features/notificationsAdmin/services/notificationsService.js] posts that payload to `admin/notification-marketing`. The client labels successful create as “sent”; actual delivery, recipient selection, scheduling timezone, and cancellation behavior require backend confirmation.

The package lookup for notification targeting fetches the full array from `GET /AdminLookup/packages` without query parameters, then performs search and pagination locally in the service. It is not a server-paginated lookup.

CHAPTER 11

Catalog and content

README · API integration · Forms and UI states

Reference and service catalogs

The dashboard exposes CRUD pages for provider categories [src/features/categoreis/service/categoryService.js], service categories [src/features/ServiceCategory/services/Servicecategoriesservice.js], services [src/features/services-admin/services/ServicesService.js], specialists [src/features/specialists/service/SpecialistsService.js], governorates [src/features/governorates/services/governoratesService.js], cities [src/features/cities/service/citeisService.js], ticket types [src/features/ticket-types/services/ticketTypesService.js], and subscription/plan types [src/features/subscription-types/services/subscriptionTypesService.js]. Their pages use feature-owned hooks, schemas, and tables. Most lists pass page number, page size, and search text. Service lists also filter by category and status; city lists filter by governorate and active state.

Category creation and update send bilingual names and image fields as multipart data. Services send category, bilingual name, optional instruction, and optional CPT fields as JSON. Subscription types send bilingual name/description, price, and discount percentage and have an active-state toggle. The meaning of prices, CPT values, and server-side deletion rules is backend-dependent.

Medicines [src/features/Medicines/services/Medicinesservice.js] have a separate list/detail and multipart create/update service. The form sends bilingual names, price, form, and an optional image; update sends an image-change flag and only sends a new image when selected. These are client payload rules, not a complete medicine data contract.

Sliders

Offer sliders [src/features/offer-sliders/pages/SlidersPage.jsx] list records with pagination and provider-name search. `sliderService.js` loads details and creates/updates multipart records with a provider ID and English/Arabic image files. Update includes image-change flags. The detail page [src/features/offer-sliders/pages/SliderDetailsPage.jsx] supports viewing and editing. Live image storage or publishing behavior is not established by the source.

Policy and contact content

Terms and conditions, refund policy, and privacy policy each have a GET and POST service using `descriptionEn` and `descriptionAr`: terms [src/features/termsAndConditions/services/Termsandconditionsservice.js], refund [src/features/refundPolicy/services/RefundPolicyservice.js], and privacy [src/features/privacyPolicy/services/PrivacyPolicyservice.js]. Their pages expose bilingual editing; `BilingualRichTextEditor` [src/shared/components/BilingualRichTextEditor.jsx] is shared UI for this kind of content.

Contact information [src/features/contactUs/services/contactsUsService] uses `GET /admin/contact-us` and `POST /admin/contact-us` with phone, email, WhatsApp, and optional social account fields. The service file currently has no extension. The backend's publication timing and external visibility of these content changes cannot be

determined from this client.